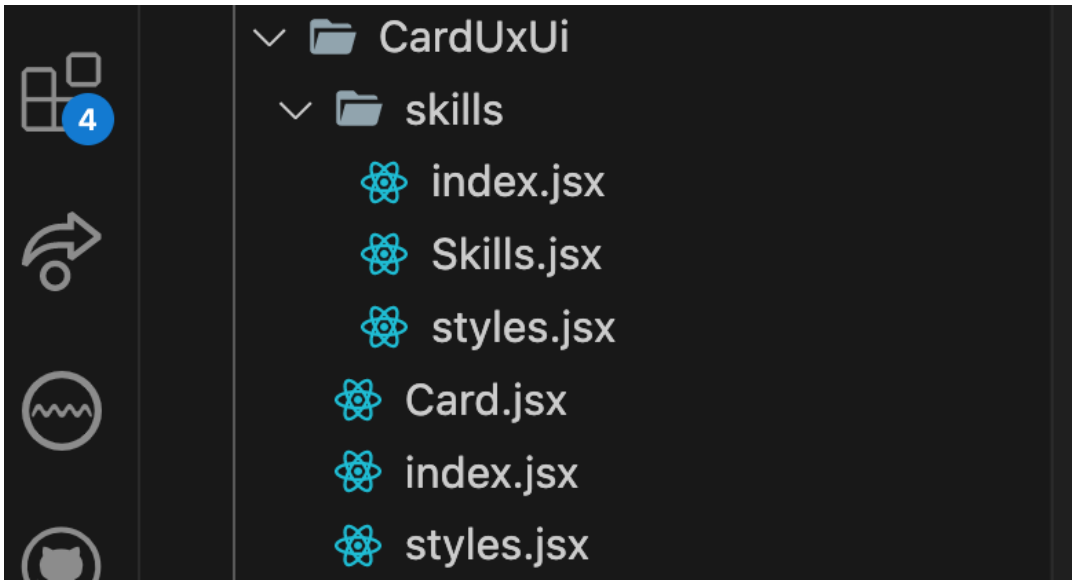
React Teorihandbok & Redogörelser
Paul Evans - UFX22
2023-06-01

1. React

React är ett open-source ramverk och ett bibliotek för Javascript. React är lätt att lära sig och lätt att använda och perfekt för ensidiga applikationer med flera komponenter. Det ger mindre kodning och ger mer funktionalitet.

En av de främsta fördelarna med att använda React JS är dess potential att återanvända komponenter. Det sparar tid för utvecklare eftersom de inte behöver skriva olika koder för samma funktioner. Detta kan göra det lättare att bygga och underhålla. Dessutom, om några ändringar görs i någon viss del, kommer det inte att påverka andra delar av applikationen. React är unikt i det faktum att de använder något som kallas separating "concerns" genom att koppla enheter inuti komponenter som innehåller både markup och logik, snarare än att placera dem i separata filer.

Ett exempel på detta i mitt eget projekt är när jag använder kort. Inuti en kortmapp har vi olika jsx-filer. Card.jsx har strukturen på kortet, vilken information som ska hamna vart på kortet. Filen och importerar stylingen från style.jsx i samma mapp.. Skills.jsx importerar data från card.jsx och kör den genom en funktion som levererar data till kortet eller samlar in data från en json-fil som innehåller global data. Som sagt, en stylingfil finns också i denna mapp vilket innebär att allt är fristående från resten av applikationen och enkelt kan redigeras utan att det påverkar resten av programmet.



React använder sig av JSX (JavaScript Extension), som är en speciell syntax som låter HTML quotes och HTML tag syntax rendera särskilda underkomponenter. Detta kan göra det lättare att förstå mellan två separata språk.

React jobbar med ett virtuellt DOM vilket gör att komplexa appar körs extremt snabbt. Mer om det virtuella Dom kommer i nästa fråga.

React JS har också vunnit popularitet på grund av närvaron av verktyg från React Developer Tools. Designad som utvecklar tillägg för Chrome och Firefox och låter dig inspektera React-komponent hierarkierna i den virtuella. Tillsammans med detta ReactJS är valet av de flesta webbutvecklare eftersom det erbjuder ett mycket rikt JavaScript-bibliotek.

Enligt min personliga erfarenhet tror jag att fördelarna överväger nackdelarna. De få nackdelar jag har läst om är att React och teknikindustrin utvecklas i väldigt hög takt vilket kan leda till begränsad dokumentation och entusiasm för att lära sig ett nytt ramverk. ReactJS täcker också bara UI-lagren i appen och inget annat. Så man behöver fortfarande välja några andra tekniker för att få en komplett verktygsuppsättning för utveckling i projektet. Och slutligen anser vissa utvecklare att jsx är för komplext och kan vara ett hinder för nybörjare.

2. Rendering och Virtual DOM?

Rendering avser att visa ut data i webbläsaren. När en funktion i React körs, renderar det ut en .jsx fil.

Ett virtuellt DOM-objekt är detsamma som ett riktigt DOM-objekt, förutom att det är en lätt kopia. Detta innebär att den inte kan manipulera element på skärmen. Vid varje ändring av en egenskap, uppdaterar den bara motsvarande noder och inte hela trädet. Det gör det till ett snabbt och effektivt alternativ. Eftersom det inte finns någon direkt ändring av DOM-elementen i Virtual DOM, leder det till smidigare och snabbare prestanda.

React tittar också bara på DOM-elementen som behöver uppdateras och utför sedan de specifika uppdateringarna istället för att åter rendera alla element. Om ett element i den verkliga DOM ändras, kommer DOM att återrendera elementet och alla dess underordnade. När det gäller att bygga komplexa webbapplikationer med mycket

interaktivitet och tillståndsförändringar är detta tillvägagångssätt långsamt och ineffektivt.

Efter att den virtuella DOM har uppdaterats jämför React den med en snapshot av den virtuella DOM som tagits precis före uppdateringen, bestämmer vilket element som ändrades och uppdaterar sedan endast det elementet på den verkliga DOM. Detta är en metod som den virtuella DOM använder för att optimera prestanda.

Tack vare den virtuella DOM behöver du inte oroa dig för state transitions. När du har uppdaterat tillståndet ser React till att DOM matchar det tillståndet.

Den virtuella DOM har möjliggjort för en snabb och enkel redigeringsprocess när jag bygger min portfölj. Det faktum att när jag renderar en komponent går den till den virtuella Dom och man kan jobba med localhost betyder att alla ändringar är nästan i realtid.

Nedan är ett exempel från min React portfolio som visar en funktion. När funktionen körs return det rendera ut det som är mellan parentes och sen är det exporterat som en .jsx fil.

```
const Icons = () => {

  return (
    <IconContainer>

      <IconContext.Provider value={{ size: "1.5em"}}>
        {footerData.map(Link => (
          <StyledIcon href={Link.href} key={Link.id}>
            {Link.icon}

            <IconDes>{Link.des}</IconDes>
        </StyledIcon>))}

          </IconContext.Provider>

    </IconContainer>
  );
}
```

3. Vad är JSX?

JSX (JavaScript Syntax Extension och ibland kallad JavaScript XML.JSX skapas av Meta (tidigare Facebook). Det liknar en annan extension syntax skapad av Meta för PHP som heter XHP.JSX tillåter oss att skriva HTML-element i Javascript och placera dem i DOM genom att konvertera HTML-taggarna till React-element utan att behöva använda andra metoder som createElement() eller appendChild(). Det gör det enklare att skriva och lägga till HTML i React.

Fler skillnader mellan HTML och JSX:

- JavaScript kan skrivas direkt. Du kan göra detta genom att sätta JavaScript i hängslen {...}. I HTML behöver du en script tagg eller en extern JavaScript-fil för att implementera JavaScript:

- För att definiera class namn och for attribut i JSX gör du det inte som class eller for. Istället skapar du class komponenter. För att definiera classnamn i JSX gör du det som "className" och för attribut för etiketter skriver du "HTMLFor":

- Alla HTML-attribut och händelsereferenser måste skrivas i camelCase när du skriver JSX.

- För att definiera inline-stilar för JSX, skriver du det som ett objekt, med egenskaperna i camelCase, värdena inom citattecken, och sedan skickar du det inline till JSX. Det betyder att du måste öppna ett style attribut och använda {} istället för citattecken. Detta är i motsats till HTML där du är fri att definiera miljontals stilar direkt inuti öppnings taggen utan att skriva dem som objekt och sätta dem inom citattecken:

Att kunna använda react-element istället för de vanliga HTML-taggarna inom mitt projekt har bidragit till enklare läsbarhet och förståelse. Samma sak med att kunna

skriva javascript direkt utan att referera till en annan fil gör att du kan se funktionaliteten direkt. Att använda anpassade element och stiliserade komponenter har också hjälpt till att bryta ner långa kodbitar i små bitar som tydligt kan märkas för att överensstämma med den styling eller den komponenten den är fäst vid.

Exemplet nedan från mitt portfölj projekt visar en kustom img-tagg "StyledImg" i taggen en javascript funktion har lagts till "onClick"..

```
<StyledImg src={PaulNavImage} alt="The dude" onClick={() => {setOpen(false);}} />
```

```
const StyledImg = styled.img`
   height: 5em;
   margin-left: 3em;
   margin-right: 1em;
   margin-top: 0.5em;

   @media screen and (max-width: 960px) {
       height: 4em;
 }

   @media screen and (max-width: 767px) {
   margin-left: 1em;
   margin-top: 0em;
   }
`;
```

4. Vad är ett undantag inom programmering?

Ett exception är en händelse, som inträffar under körningen av ett program, som stör det normala flödet av programmets instruktioner.

När ett fel uppstår inom en metod skapar metoden ett objekt och lämnar det till runtime-systemet. Objektet, som kallas ett exception objekt eller undantag objekt, innehåller information om felet, inklusive dess typ och programmets tillstånd när felet inträffade. Att skapa ett exception objekt och lämna det till runtime-systemet kallas Throwing an exception (kasta ett undantag).

Efter att en metod throws an exception, försök runtime-systemet att hitta något för att hantera det. En ordnad lista som kallas en call stack med möjliga lösningar eller handlers skapas.

Runtime-systemet söker i call stack efter en metod som innehåller ett kodblock som kan hantera undantaget. Detta kodblock kallas en exception handler. Sökningen börjar med metoden där felet uppstod och fortsätter genom call stack i omvänd ordning som metoderna anropades. När en lämplig handler hittas skickar runtime-systemet undantaget till hanteraren. En exception handler anses lämplig om typen av det kastade exception object matchar den typ som kan hanteras av hanteraren.

Om det runtime systemet söker igenom alla metoder i call stack utan att hitta en lämplig exception handler avslutas runtime systemet och programmet.

Att använda undantag för att hantera fel har vissa fördelar jämfört med traditionella fel hanteringstekniker. Två fördelar är; att skilja felhanteringskod från vanlig kod och att gruppera och differentiera feltyper.

Undantag ger möjlighet att separera detaljerna om vad man ska göra när något utöver det vanliga händer från huvudlogiken i ett program. I traditionell programmering leder feldetektering, rapportering och hantering ofta till förvirrande kod.

Gruppering eller kategorisering av undantag är ett naturligt resultat av klass hierarkin eftersom alla undantag som kastas inom ett program är objekt.

Man vill att exception handlers ska vara så specifika som möjligt. Anledningen är att det första en handler måste göra är att avgöra vilken typ av undantag som inträffade innan den kan besluta om den bästa återställnings strategin. Annars måste hanteraren tillgodose alla möjligheter. Exception handlers som är för generella kan göra koden mer felbenägen genom att fånga och hantera undantag som inte förutsågs av programmeraren och som hanteraren inte var avsedd för.

Ett exempel på en specifik exception handlare

FileNotFoundException betyder att en fil inte kunde hittas på disken.

5. Vad innebär autentisering inom webbapplikationer? Vad används det till?

Autentisering är processen för att verifiera en användares identitet. Se till att en användare är den de säger att de är.

Auktorisering säkerställer att en inloggad användare har rätt att utföra specifika åtgärder eller se viss data. Till exempel ska jag kunna redigera min egen personliga information, men jag borde inte se någon annan användares data.

Både autentisering och auktorisering krävs för att en applikation ska vara säker.
Du kan implementera autentisering med en eller flera av följande metoder:

- Vad en person vet (lösenord eller lösenfras).
- Vad en person har (engångs token eller fysisk enhet).
- Vad en person är (biometri, fingeravtrycksläsare, ansiktsigenkänning).

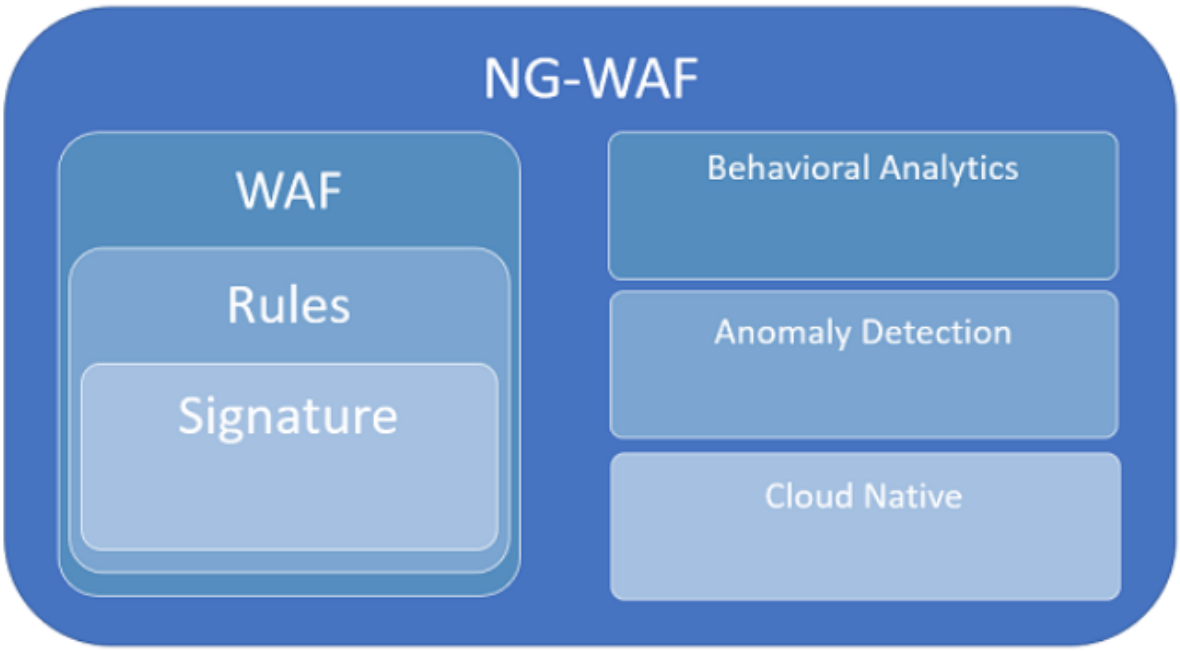
Webbapplikationer var en gång monolitiska vilket innebär att de existerade som en bitkod som kördes på en server. Servern gjorde det mesta av jobbet och skickade allt till webbläsaren på en gång. Autentiserings- och auktorisering mekanismen på en sådan webbplats är enkel. Efter att användaren loggat in på webbplatsen, verifierar en databas med användarinformation deras identitet. En session skapas på servern och alla efterföljande förfrågningar använder sessionen för att identifiera användaren utan att ytterligare inloggning krävs.

Webbapplikationer består nu av mikrotjänster distribuerade i olika moln. Varje mikro tjänst är en fristående server och datalager separat från applikationens övriga funktioner. En klientapplikation, den som användaren interagerar med, gör API-anrop till de tjänster den behöver för att göra sitt jobb. Autentisering och auktorisering ser helt annorlunda ut under denna nya distribuerade modell. Eftersom varje mikro tjänst har ett datalager har en session skapad i en ingen betydelse för en annan. API-anrop skulle ständigt gå sönder om applikationen var beroende av ett enda sessions-ID skapat av den första servern.

Distribuerade API:er förändras snabbt och behöver nya metoder för säkerhet. Autentiseringen och auktoriseringen måste också fungera för alla programmeringsspråk.

Därav NG-WAF (nästa generations brandvägg)
Nästa generations WAF har ofta definierats med tre huvudsakliga egenskaper:

- Detektering baseras på applikationen och angriparens beteende.
- Går bortom signaturer för att identifiera anomalier och grundläggande attacker.
- Molnbaserad implementering som stöder olika molnplattformar, API:er och mikrotjänster arkitektur.



Ändå är NG-WAF inte ett 100% försvar mot hackare. För att heltäckande skydda dina molnbaserade appar bör du leta efter lösningar som också erbjuder:

- End-to-end distribuerad spårning och synlighet, vilket ökar noggrannheten och djupet av attack detektering.
- Möjligheten att blockera missbruk av affärs användningsfall, insider attacker och hot aktörer, även i distribuerade system.